PAYPAL AI HACKATHON 2026

TECHNICAL SUBMISSION GUIDE

Market Metro

Agentic Commerce That Learns

From shopper behavior to a tested campaign, with ontology verification and PayPal payment evidence.



Observe

Market + warehouse

Decide

Planner + verifier

Trace

PayPal + Studio

Lanz Vincent Vencer

12 November 2026 / Submission edition 1.0

Open the demo Browse the public source

CONTENTS

A guide to the submission

Start with the walkthrough. Follow the architecture into the decisions, then inspect the evidence and reproduce the checks.

01 Submission at a glance 3
02 The judge walkthrough 4
03 System architecture 5
04 Execution modes and responsibilities 6
05 The economy and its assumptions 7
06 Warehouse, identity and lineage 8
07 Ontology: meaning before measurement 9
08 Choosing training data and features 10
09 Planner workflow and checkpointing 11
10 Financial verification and model selection 12
11 One campaign, from evidence to purchase 13
12 PayPal at the center of the value loop 14
13 PayPal API integration and lifecycle states 15
14 AG Studio: query, verify and publish 16
15 Intelligence and observability 17
16 Metrics and the featured result 18
17 Ontology benchmarks: compatibility and learning 19
18 Fresh planner versus planner with ontology 20
19 Does the improvement repeat? 21
20 Scaling to a larger shared market 22
21 Local setup and repository map 23
22 Cloud Run, security and cost controls 24
23 Verification and reproducibility 25
24 Limitations and the next validation steps 26
25 Evidence index 27
26 Useful links and credits 28

CLICKABLE NAVIGATION

Contents, source references and product links are clickable. Landscape pages give the architecture figures enough room; the PDF outline also lists every chapter.

CHAPTER 01

Submission at a glance

Market Metro is a commerce policy lab where an AI planner learns from merchant activity, tests marketing campaigns, and connects selected purchases to verified PayPal Sandbox receipts. The submission makes the path from observation to outcome inspectable.

The business problem

Merchants have browsing, inventory and payment records, but turning those records into a useful campaign requires several different decisions: what demand exists, which records can train a model, whom to contact, what to recommend, and whether the campaign creates retained commerce after costs. A persuasive explanation alone cannot establish that a campaign is worthwhile.

Market Metro provides a controlled environment for that workflow. Seeded shoppers generate activity, local tools fit models, an LLM proposes and revises campaigns, and deterministic checks control publication. The ontology connects business definitions to data. PayPal supplies native payment evidence. AG Studio exposes the decisions and their supporting records.

Submission component	What is implemented
Visual economy	Three.js market; seeded shoppers, finite budgets, stock, deliveries and refunds
Campaign intelligence	Gemini planning, local Python model fitting, development trials and checkpointed execution
Ontology	Checkout identity, money units, relationship validation and prediction-time feature eligibility
Financial control	Deterministic selection of an eligible tested campaign or the incumbent policy
PayPal	Sandbox checkout, saved consent and linked receipts; separate lifecycle integrations
AG Studio	Native queries, two-role investigation, verified dashboard publication, grids, charts and custom widgets

> **Featured evidence:** +$114.96 modeled PayPal net, +$1,024.86 merchant contribution and 16 additional retained purchases versus fixed rules. These are the results of one identified simulated campaign, not production PayPal profit.

How to read this guide

For a quick review, follow the judge path, architecture, campaign trace and results. For implementation review, continue through the ontology, harness, PayPal, AG Studio and deployment chapters. Separate studies retain their own seeds, denominators and costs.

Evidence: README, saved judge campaign, judge testing guide.

CHAPTER 02

The judge walkthrough

No account, API key, credit card or PayPal login is needed. Open the hosted landing page, select **Try the demo**, and choose **Play walkthrough**. The recorded campaign starts at day 1.

Stop	What to do	What it demonstrates
Market	Start at day 1; follow organic shopping	The environment creates observations before the review
Agent decisions	Inspect day-31 evidence, training choices and trials	What the LLM knew, what it requested and what the verifier changed
Shopper outcome	Follow Shopper 179 on days 35 and 36	Recommendation, purchase, delivery and retention belong to one trace
Impact	Compare fixed rules with the accepted campaign	Retained purchases, merchant contribution and modeled PayPal net
PayPal receipt	Open the final receipt	The existing $96 order matches a native Sandbox capture

Continue into the evidence

Open Agent Studio and choose **Replay a verified investigation**. The saved Gemini choices drive native Studio query, verification and publication again. Inspect **Tool calls & checkpoints**, **Framework conversation**, and the five published views.

Open Intelligence for model training, agent decisions, financial comparisons, LLM usage, lifecycle evidence and ontology associations. **Arrange** edits the dashboard layout; it does not alter transactions or outcomes.

Try a new market

The free simulation settings run a new 12-merchant, 600-shopper seed using the recorded model and campaign. This tests behavior on another seed without fresh model or payment calls. Outcomes can differ.

> **Replay boundary:** playback does not retrain models, request a new campaign, charge a buyer, or create a second receipt. Optional fresh Agent Studio questions use a separate shared model allowance; saved replay remains the fallback.

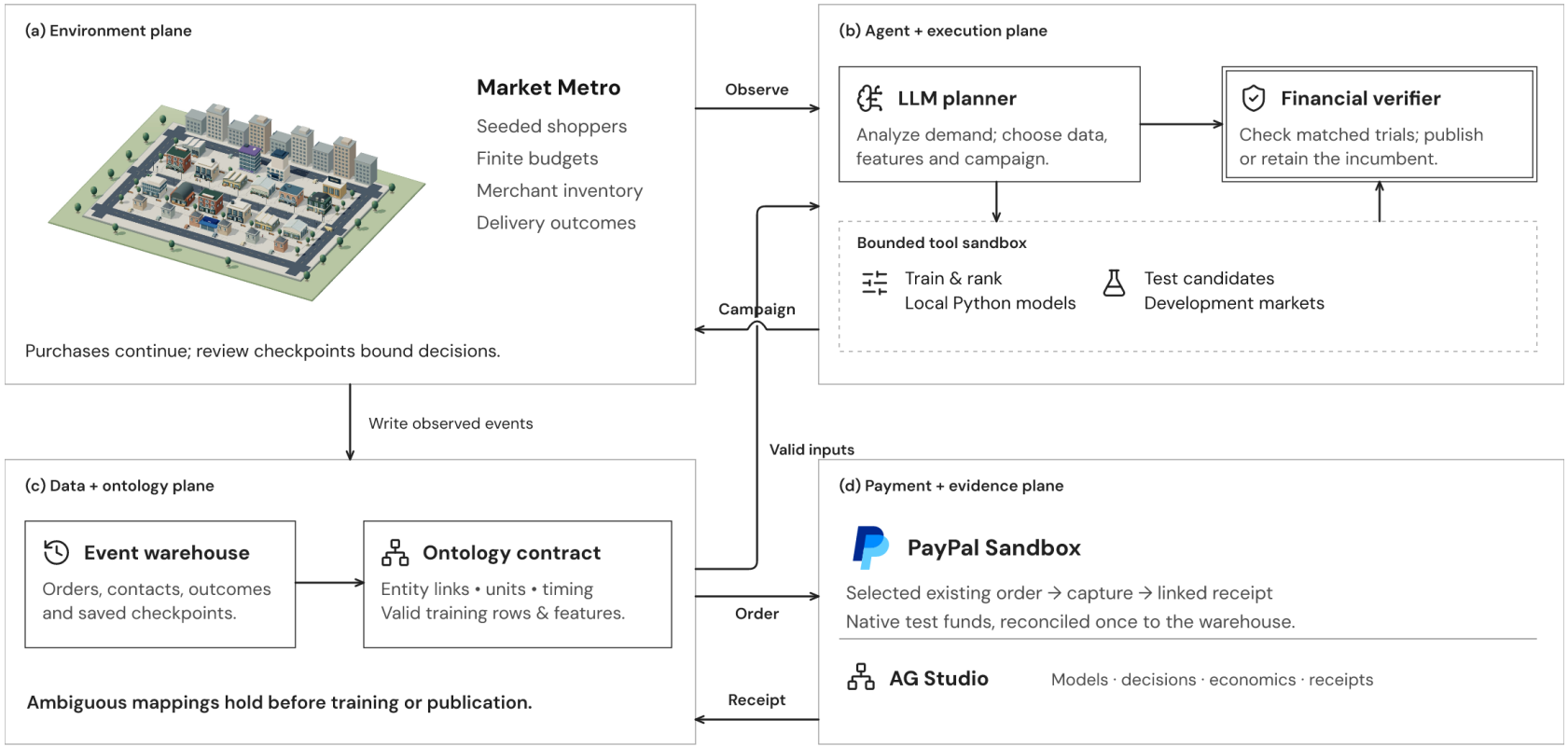
If the shared service is busy, retry later or run the repository locally. New hosted sessions are temporary; the bundled judge evidence survives restarts.

Evidence: testing instructions, public runtime boundary.

CHAPTER 03

System architecture

Four planes connect the visual market to checked decisions and payment evidence. Economic time pauses at review checkpoints while the planner analyzes, trains and tests; accepted decisions affect subsequent market activity.



Read the arrows: observed market events enter the warehouse; certified relationships constrain the planner's inputs; bounded tools train and test; the verifier releases a campaign. A selected existing order is mirrored to PayPal Sandbox, then its receipt returns to the warehouse. AG Studio inspects these records and runs its own scoped investigation.

Evidence: campaign controller, ontology training tools, Studio intelligence adapter.

CHAPTER 04

Execution modes and responsibilities

The project has several execution modes. Keeping them explicit is essential: a saved replay, a fresh investigation and a fresh economic experiment establish different things.

Mode	LLM work	Market / payments	Intended use
Guided judge campaign	Recorded six-decision review	Recorded outcomes and existing Sandbox receipt	Explain the complete campaign
Free new-seed run	Reuses saved model and policy	New numerical market; no native payment	Explore the frozen economy
Agent Studio replay	Reuses two recorded role choices	Native query, validation and view publication	Inspect the sponsor integration
Fresh Agent Studio	Two bounded Gemini decisions	Read-only economic records; dashboard updates	Ask a new cohort question
Configured local campaign	Fresh Gemini review and local model fits	New market; optional explicitly enabled Sandbox sample	Reproduce or extend experiments

Where the intelligence lives

The campaign planner chooses analytical focus, a training window, model experiments and a campaign proposal. Local Python performs fitting. The financial verifier owns the final publication decision. The market engine owns customer behavior and settlement.

The separate AG Studio workflow has a **commerce planner** and an **insight editor**. Native query and delegation connect them; deterministic verification and publication constrain them. This investigation can update dashboards, but cannot rewrite campaign results or execute payments.

Why the default is one campaign planner

The six-seed comparison gave one planner and a three-role team the same evidence, tools, decision budget and verifier. Both improved over fixed rules in aggregate. One planner produced higher aggregate PayPal net, merchant contribution and purchases. The default follows that evidence; extra role labels are not presented as an automatic financial advantage.

> **Control boundary:** the LLM proposes within an allowed interface. It cannot issue arbitrary SQL writes, invent funds, alter future shopper needs, publish an untested campaign, or supply an arbitrary PayPal recipient.

Evidence: six-seed comparison, Studio role boundaries.

CHAPTER 05

The economy and its assumptions

The featured economy contains **12 authored merchants and 600 simulated shoppers**. Organic activity supplies the first 30 days of observations. The displayed day-31 review uses only that observed history and eligible historical training markets. The campaign runs through market day 100, followed by settlement through day 126.

What the environment models

- Shoppers have finite budgets, prior interests, consent and dated needs. Recommendations and general needs inquiries are distinct actions.
- Merchants have inventory, cash, replenishment choices and fulfillment obligations. A recommendation cannot create stock or shopper money.
- Orders can span several merchants. Checkout parents and seller portions are recorded separately.
- Payment, delivery, refund and settlement states affect retained outcomes. Immature outcomes do not become retained-purchase labels.
- Contact budgets and outreach costs constrain campaigns. More contacts or more clicks do not necessarily create more net value.

Fixed inputs, changing decisions

Within a comparison, policies receive the same initial state and exogenous shopper needs, consent and funding conditions. Policy actions may change outreach, ranking, replenishment, fulfillment and the resulting trajectory. The simulator is held fixed for that comparison; it is not adjusted afterward to force a favorable result.

Review dates pause while analysis, fitting and trials run. Once the controller publishes or retains a policy, dates resume. This gives a bounded demonstration and avoids stale decisions caused by a fast simulation clock outrunning local computation.

What these assumptions cannot establish

The catalog and behavior are authored fixtures. Neither PayPal Sandbox nor the current application provides PayPal's internal consumer histories. The current candidate makes no Channel3 calls. A positive result measures performance inside the stated environment; external merchant outcomes require calibration and controlled real-world evaluation.

The larger **commerce-metropolis-1** profiles are separately versioned markets. Their population and execution details are documented independently; their absolute gains are not combined with the 12-merchant result.

Evidence: featured experiment scope, shared-market assumptions, frozen comparison protocol.

CHAPTER 06

Warehouse, identity and lineage

SQLite stores the economic events and decision artifacts. The visualization and intelligence workspace read projections of these records. Monetary fields use integer cents; simulation dates and native provider timestamps retain their distinct meanings.

Record family	Grain and purpose
Runs, merchants, products, customers	Stable identities and the authored initial state
Events and contacts	Observed behavior or one outreach action, with day and source
Checkout parents	One shopper checkout, even when several merchants fulfill it
Orders and order items	One seller portion and its item-level detail
Transactions, refunds, deliveries	Financial and fulfillment lifecycle observations
Inventory movements	Stock changes linked to orders and replenishment
Models, calls and checkpoints	Fitted artifacts, provider usage, proposals and verified decisions
PayPal receipts / operations	Native provider evidence linked to existing warehouse orders

The critical join

The ontology exporter reconstructs checkout-to-order relationships from **autonomy_baskets.legs_json**. It cross-checks each order's run, merchant, shopper and amount before releasing the portable feed. Summing a checkout total after joining it to multiple seller or item rows would overstate commerce.

The full judge-run capture ledger contains **1,384 unique checkouts**, **1,922 merchant orders** and **538 split checkouts**, with **$164,451 captured gross**. These totals include warmup and purchases subsequently refunded. They are not the same metric or period as 643 retained campaign purchases.

Data products for inspection

The warehouse exposes marketing transactions, daily merchant sales, campaign performance, funnel activity, inventory and PayPal order lineage. The Intelligence adapter counts checkout parents directly and aggregates merchant portions separately. Missing model metrics remain missing; pooled and product-contact validation scores retain their cohort labels.

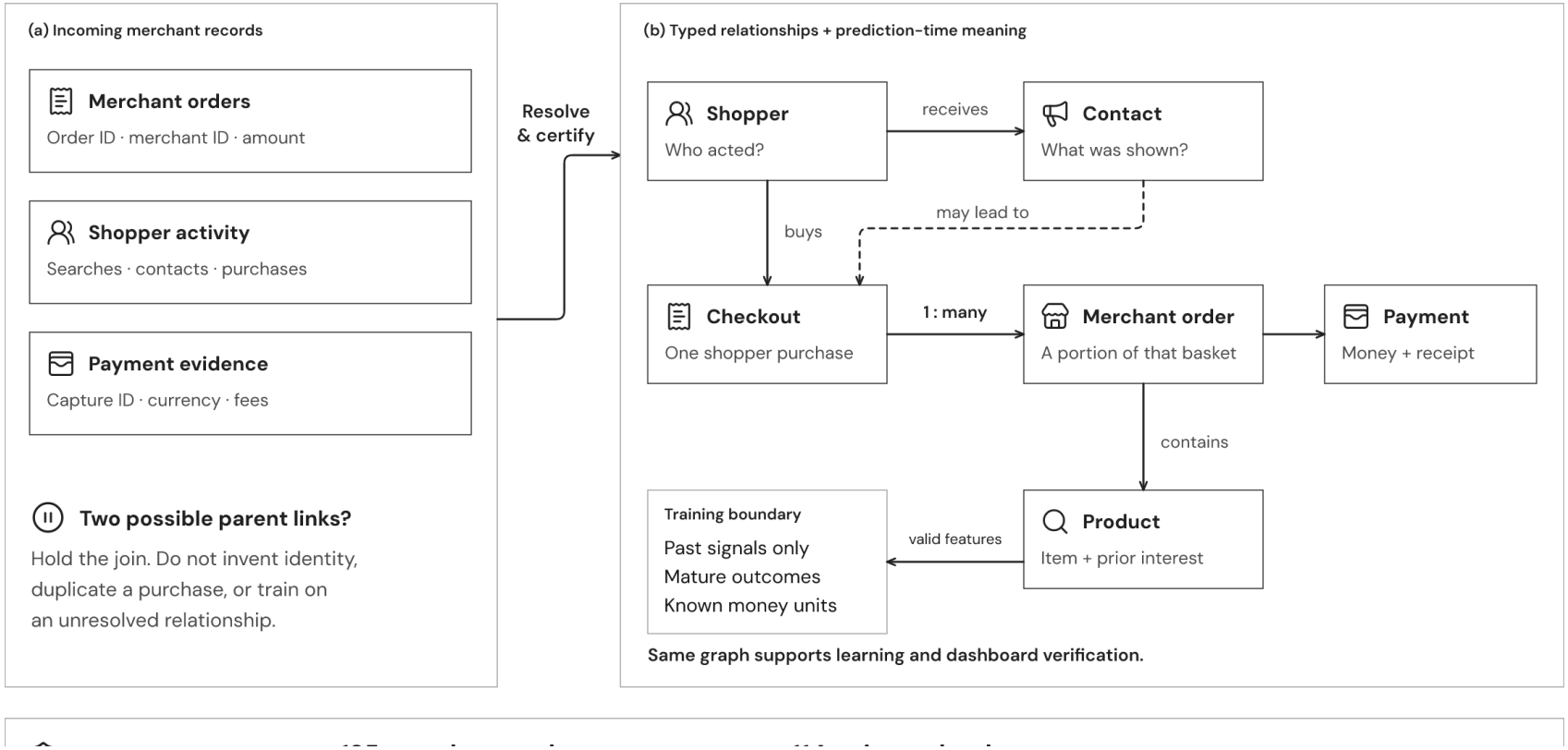
> **No double credit:** a native Sandbox receipt corroborates an existing order. It contributes zero additional simulated purchases and zero additional modeled gross.

Evidence: warehouse schema, ontology exporter, Intelligence adapter, portable judge records.

CHAPTER 07

Ontology: meaning before measurement

The project-owned ontology binds business terms, entity relationships, units and observation timing to merchant records. NVIDIA Auto Ontology informed the design; it is not an installed runtime dependency or an automatic schema-discovery service.



CERTIFY releases measurements supported by the declared relationships and accounting checks. **HOLD** withholds totals or training inputs when meanings conflict. On the day-31 example, two declared checkout links cause a hold; correcting the dictionary admits 715 checkouts and restores a 36-contact campaign preview without changing any underlying row.

Evidence: ontology implementation and limits, planner gate, hold/resume example.

CHAPTER 08

Choosing training data and features

The ontology does more than rename columns. It constrains which business question a model can answer, which rows belong to that question and what information was available at decision time.

Business question	Admissible data / response
Who will retain a basket after a product recommendation?	Product-naming contacts, mature retained-basket labels and pre-contact features
Who may answer a general needs inquiry?	Separate reply and settled-value model; no fabricated SKU features
Will the shopper keep this exact item?	Hold: the historical export lacks fulfilled-basket item identities
Which campaign creates more value?	Matched numerical trials and settled financial outcomes

A concrete training contract

The 28-day historical window contains **2,762 mature contacts** across seven historical markets: **1,230 product recommendations** and **1,532 general inquiries**. Of the product contacts, 116 led to retained purchases. Labels mature after the modeled 26-day settlement window.

Eligible relationships expose prior product interest, shopper history and earlier market responses. A current contact's response, authorization or refund can be a label or evaluation outcome, but cannot become a pre-contact predictor. Feature names are checked for eligibility and duplication; the selected set must include prior intent and shopper-history evidence.

The live ontology interface lets Gemini request the exact field list. The local tool fits those fields, records the dataset and model hashes, and checks Python-to-JavaScript prediction parity. The financial verifier then compares the fitted candidate with its alternatives.

A useful rejection

In the fresh 12-merchant ontology run, Gemini selected eight features. Their recommendation ROC AUC was **0.7311**, below **0.7668** for the broader candidate. The workflow retained the broader 20-field model and revised the campaign instead. This is evidence of constrained experimentation and rejection, not proof that every ontology-selected feature set improves prediction.

Evidence: training study, training contracts, LLM feature-selection interface, fresh comparison.

CHAPTER 09

Planner workflow and checkpointing

The campaign review is a bounded sequence of decisions rather than an unrestricted chat loop. Tools return measured artifacts. The LLM chooses what to investigate or revise; code controls data access, execution and publication.

Stage	LLM decision	Trusted operation / saved artifact
Observe	Choose a demand hypothesis and analytical focus	Snapshot, category evidence and eligible history
Prepare	Choose a window, feature view or ontology target	Training contract, cutoff and dataset fingerprint
Train	Request two model experiments	Local fits, validation reports, model bytes and parity checks
Propose	Select a model and campaign configuration	Typed candidate bound to evidence and incumbent policy
Test	Inspect development-market outcomes	Exact candidate trials, money and purchase checks
Revise / finish	Change the proposal or end the search	Valid revision or completed review; controller selects publication

Harness guarantees

The owned schemas generate tool declarations and validate replies. Required fields, enums, types, nesting, bounds and evidence references are checked without implicit coercion. Unknown actions and fields are rejected. Rationale is stored as an explanation, never interpreted as executable instructions.

Review artifacts bind snapshot, date, market profile, prior policy, model bytes, campaign and trial context. Repeated same-day planning shares an in-flight operation; changed evidence, overlapping dates and rewinds are rejected. A repeated identical candidate can reuse its recorded trial.

Recovery behavior

Bounded retries apply only to recognized safe failures. An uncertain provider POST is not blindly repeated with a new identity. Local numerical timeouts may reuse the exact saved input within retry limits; incomplete or invalid outputs cannot authorize publication. A rejected optional revision leaves earlier completed trials available.

Checkpointing is not a claim of universal crash recovery. The deployed service can mark uncertain runs interrupted, and hosted sessions use temporary storage. Model output remains nondeterministic; the validation and selection rules are deterministic for the same validated inputs.

Evidence: typed-execution design, controller, contracts.

CHAPTER 10

Financial verification and model selection

The LLM has **no direct campaign-publish tool**. A deterministic controller considers the tested initial proposal, an incumbent-supply variant when applicable, and valid revisions. The incumbent remains available when no candidate qualifies.

Three layers of checks

Layer	Requirement
Structure	Valid typed proposal, allowed values and bounded inputs
Evidence	Exact model, market, snapshot and candidate identities match complete trial records
Business	Validation, PayPal contribution, merchant contribution, purchases, volume, cash, inventory and liability checks pass

Eligible candidates are ordered by measured modeled PayPal contribution, then merchant contribution, then retained purchases, with a canonical contract hash as the stable tie-break. A worse revision cannot erase a better eligible earlier candidate.

Why ROC AUC is not the final objective

The featured planner preferred a two-stage response model. On recommendation-contact validation it scored about **0.7992 ROC AUC**, versus **0.7668** for boosted trees. Yet the matched campaign trials averaged **+$26.65** modeled PayPal net over fixed with the response model and **+$30.97** with boosted trees.

The boosted candidate also cleared the purchase and merchant checks. The verifier therefore bound that model to the published campaign. A ranking metric measures discrimination on a particular label and cohort; financial outcomes also depend on which actions fit stock, budgets, timing, fulfillment and costs.

Development is not evaluation

The normal review uses three separate development markets covering discovery, low response and no response. It does not reveal the active market's future. Final campaign evaluation occurs after publication on the matched evaluation seed. The large-market profiles use 12-merchant proxy trials and a full-size final comparison; proxy approval alone is not evidence of scale-wide benefit.

> **Guardrail limit:** deterministic validation provides controlled execution and evidence binding. It cannot guarantee profit on unseen shoppers or validate the simulator against reality.

Evidence: verifier rules, model-check implementation, recorded choice.

CHAPTER 11

One campaign, from evidence to purchase

Reference: seed 92111706; 12 merchants; 600 shoppers; recorded review cost $0.032325. The six model decisions and two fitted models are saved with the campaign.

Time	Evidence or action	Consequence
Days 1-30	Organic shopping generates the current history	No future campaign outcomes enter the review
Day 31	99 electronics and 85 furniture interest signals; 6 and 35 open funded units respectively	Planner investigates category demand
Day 31	Historical pool: 7,010 contacts, 457 retained purchases, including 41 with delayed funding	Planner chooses a 28-day feedback window and two fits
Day 31	Ranked discovery; up to 36 contacts per day	Recommendations and needs inquiries remain distinct
Day 31	Verifier changes the preferred response model to boosted trees	Published campaign is tied to the tested alternative
Day 35	Shopper 179 receives a Mobile Side Table recommendation	Buys from Form & Function for $96
Day 36	Delivery completes and the purchase is retained	Order can contribute to retained commerce
Day 126	Campaign liabilities settle	Compare final outcomes with the fixed policy

The shopper's previous purchase was on day 19: **16 days earlier**. The same request remained unfilled under fixed rules and the planner's original ranking. The selected trace demonstrates one concrete effect of the verified model change.

Payment lineage

The existing warehouse order was mirrored once to the connected PayPal Sandbox test store. Its completed capture is **2M324038C71499318**. Provider readback reconciles **$96.00 gross = $3.56 fee + $92.44 test-merchant proceeds**. The order's custom ID matches the warehouse order.

Replaying this story reads the saved receipt. It does not issue a new payment or count another sale. Provider timestamps remain separate from the simulated day-35 event.

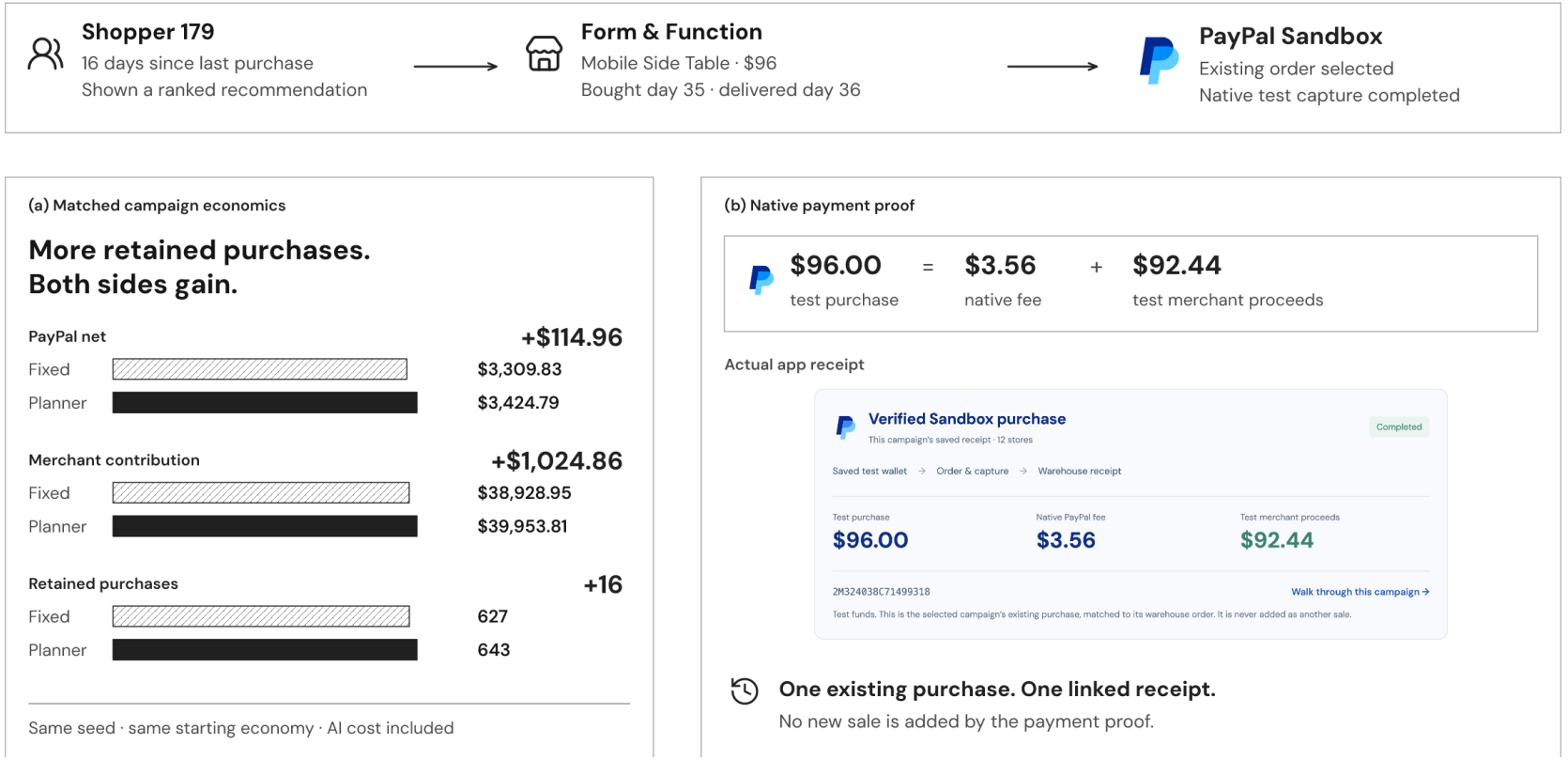
This seed was selected from the six-seed study because it contains the single-planner verifier's model switch. It is an explanatory case, not an unbiased estimate of expected production lift.

Evidence: complete trace and receipt, walkthrough method.

CHAPTER 12

PayPal at the center of the value loop

PayPal has two explicit roles: the modeled payment economics shape the campaign objective, and native Sandbox APIs verify execution of selected existing orders. The native receipt and the simulated policy comparison remain separate evidence types.



Value proposition: test policies that produce retained, fulfillable purchases; measure fees, processing assumptions, refunds and costs; then trace selected payment execution to the same order. The POC demonstrates this workflow. It does not claim access to PayPal internal data, control of PayPal risk policies, or proven production profit.

Evidence: judge receipt, native service record, financial comparison scope.

CHAPTER 13

PayPal API integration and lifecycle states

All native operations use **PayPal Sandbox** and fictitious funds. OAuth and credentials stay server-side. The primary $96 receipt is separate from the earlier $31.99 lifecycle trial.

Service workflow	Implementation purpose	Recorded state
Payment Method Tokens / Vault v3	Reuse a test wallet after initial buyer consent	Existing saved wallet verified; private token not exported
Orders / Checkout v2	Mirror an observed purchase and verify capture	Completed capture with fee and proceeds
Payments v2 refunds	Apply an approved packing-defect adjustment	$1 partial refund completed in the lifecycle trial
Invoicing v2	Create an itemized two-unit procurement invoice	$63.98 UNPAID; notification emails suppressed
Catalog Products + Subscriptions	Create a finite replenishment offer	One $31.99 monthly cycle; APPROVAL_PENDING, never activated
Shipment tracking via Orders v2	Link fulfillment metadata to a captured order	SHIPPED accepted and read back; delivery remains simulated

These are six verified service workflows, with different success meanings. An unpaid invoice is not a sale; an unapproved subscription is not recurring revenue. Tracking acceptance is not independent proof of physical delivery.

Unavailable paths remain visible

Payouts returned **INSUFFICIENT_FUNDS**; no cashback was delivered. Transaction Search returned **NOT_AUTHORIZED**. Neither is claimed as a successful integration. The application uses authenticated readback; a production webhook implementation is not claimed.

Execution controls

Recipients and permitted amounts are resolved server-side. Operations are persisted before provider requests and reuse stable request identities. Uncertain creation outcomes stop for reconciliation. Native samples link to existing orders, and refunds or pending obligations keep their actual states. Buyer approval is necessary for initial reusable-wallet consent; a separate subscription needs its own consent.

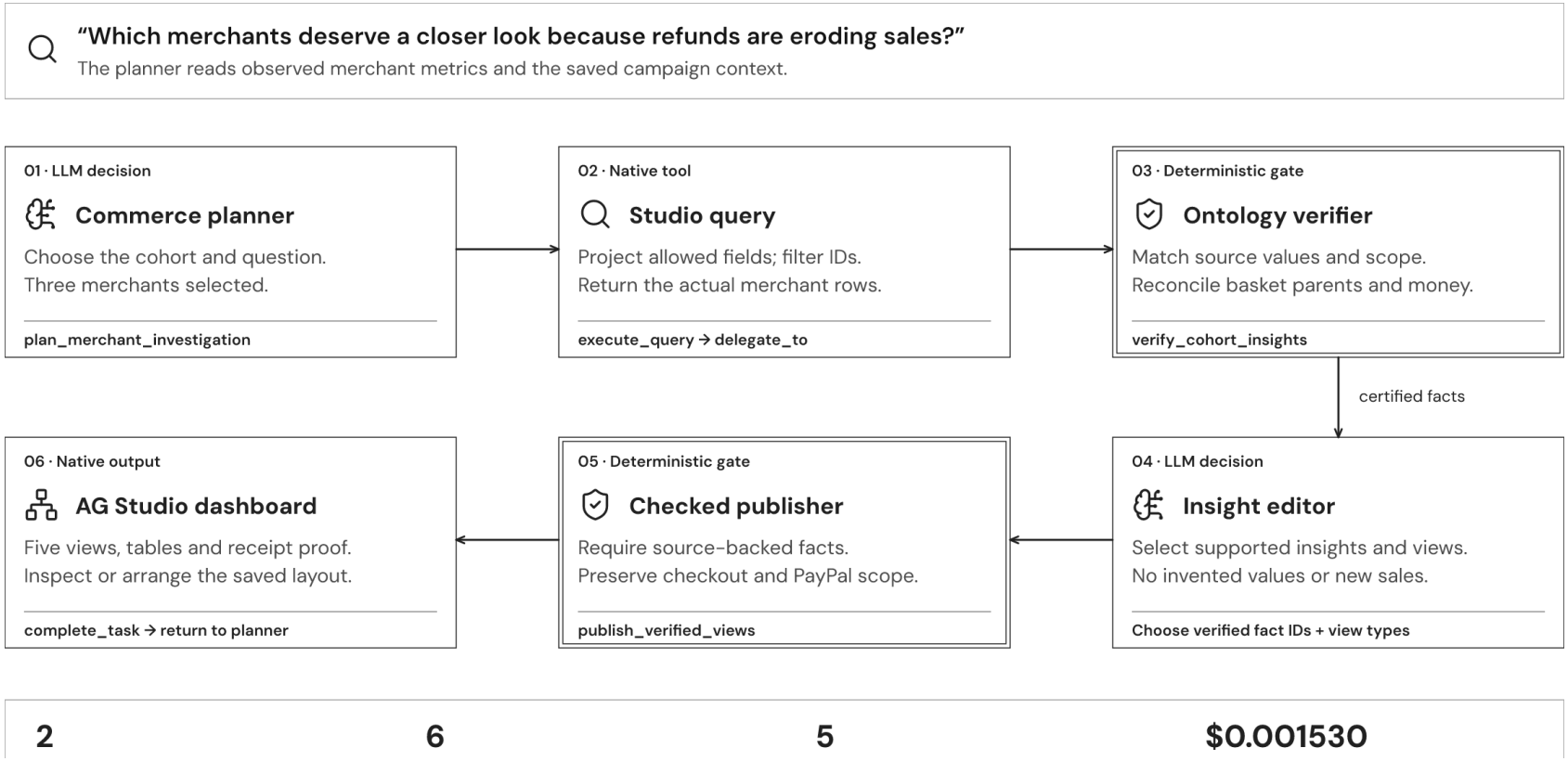
The hosted judge service disables new payment creation and exposes existing proof. Local configured modes can exercise authorized Sandbox flows; live PayPal mode is unsupported.

Evidence: service guide and native IDs, Sandbox setup, payment-lineage schema.

CHAPTER 14

AG Studio: query, verify and publish

The sponsor workflow turns the dashboards into an investigation workspace. Two Gemini roles make bounded decisions; Studio's native tools and the ontology connect them to actual recorded merchant data.



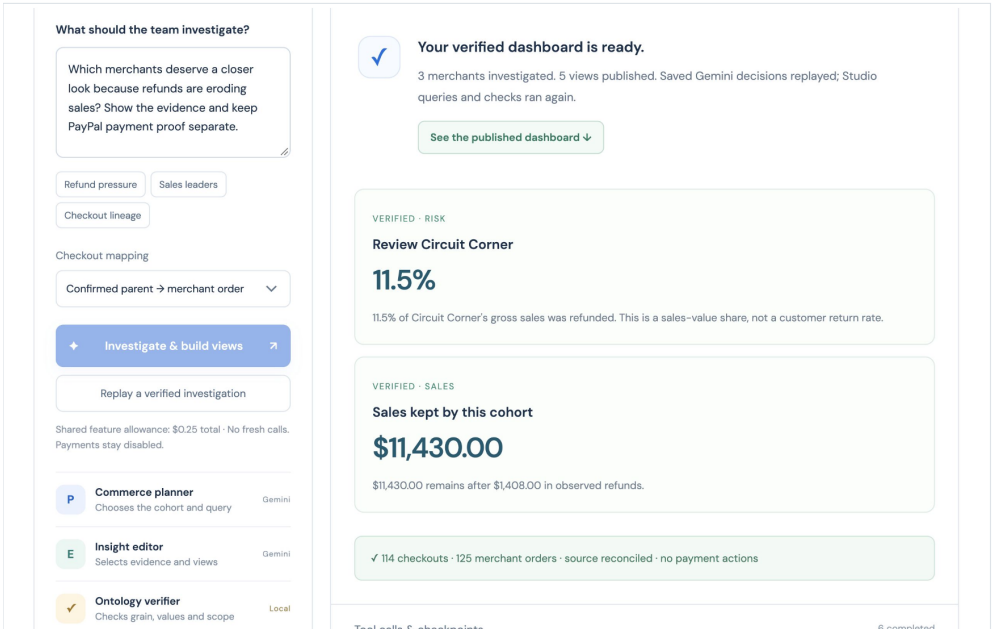
The saved question selects Circuit Corner, The Bundle Shop and Paper Lane. Verification reconciles **125 merchant orders to 114 checkouts**, with **$12,838 gross sales - $1,408 refunds = $11,430 retained sales**. Two model calls cost an estimated **$0.001530**; six native tool calls publish five views. Retained sales are not profit, and the separate $96 receipt is whole-market integration evidence.

Evidence: Studio integration, native harness, recorded investigation.

CHAPTER 15

Intelligence and observability

AG Studio hosts the analytical workspace; AG Grid provides sortable and filterable records, and AG Charts visualizes measured outcomes. DM Sans, the shared blue navigation and custom widgets keep the workspace consistent with the market.



Workspace view	What a judge can inspect
Market trends / overview	Demand, category spending, checkout volume and merchants
Model training	Cohort-specific ROC AUC, log loss, features, rows and local fit duration
Agent decisions	Day and stage, supplied evidence, proposed action, checkpoint and publication
LLM observability	Provider model, input/output usage, latency, status and estimated cost
Verification / financials	Candidate trials, rejected checks, fixed comparison and contribution
PayPal / lifecycle	Native receipt, capture state, fees and order linkage
Ontology	Clickable relationships, checkout parents, seller portions and feature associations

Arrange uses Studio's native layout editor. Custom PayPal and certified-count widgets use the same source records. The decision and ontology graphs are application components; they are not represented as native AG Charts graph types.

The intelligence dashboards have defined default layouts. Agent Studio's separate two-role workflow chooses supported facts and views through **createAiHarness**, native **execute_query**, **delegate_to**, checked publication and **complete_task**. It is not a claim that Studio's default built-in agents independently made the campaign decisions.

Evidence: workspace implementation, agent workspace.

CHAPTER 16

Metrics and the featured result

The objective is retained, fulfillable commerce after modeled costs. Prediction quality, attributed purchases and financial gain are reported separately.

Metric	Meaning
Modeled PayPal net after AI	Settled modeled platform contribution, including the evaluator's payment/outreach costs, less run-specific estimated inference usage
Merchant contribution	Change in modeled merchant equity, including inventory value and obligations; not all realized cash
Retained purchases	Checkout purchases kept after the modeled refund/settlement window
Payment volume	Campaign-period payment volume; not profit
ROC AUC / log loss	Prediction quality on a named cohort and outcome; not campaign lift
Native Sandbox proceeds	Provider-reported amount for the selected test capture; not the simulated net metric

Matched campaign: seed 92111706

Settled outcome	Fixed rules	Planner + verifier	Difference
Modeled PayPal net after AI	$3,309.83	$3,424.79	+$114.96
Merchant contribution	$38,928.95	$39,953.81	+$1,024.86
Retained purchases	627	643	+16
Payment volume	$90,683	$93,028	+$2,345

The PayPal net gain is **3.47%** relative to fixed rules in this run. Six recorded decisions cost **$0.032325** by metered-token estimation. Both policies share the initial economy and future exogenous inputs; settlement extends to day 126. The native receipt does not increase either arm's financial score.

Changing only the planner's ranking through the verifier added **$21.04 modeled PayPal net**, **$325.12 merchant contribution** and **two retained purchases** compared with its original choice. That incremental verifier result differs from the entire campaign's gain over fixed rules.

> **Interpretation:** the featured campaign improved all three primary outcomes. One selected scenario does not establish a production effect, universal advantage or guaranteed result on another seed.

Evidence: exact saved values, method and selection.

CHAPTER 17

Ontology benchmarks: compatibility and learning

Two separate local studies evaluate ontology behavior. Neither created new native payments or made paid model calls.

24 held-out merchant exports

Fixed mapping and ontology binding received identical inputs and used the same accounting checks. The fixed mapper supports familiar fields, common aliases, reordered columns and a conventional flat export. Ontology binding uses declared business meanings to resolve unfamiliar physical names.

Measure	Fixed mapping	Ontology
Usable feeds certified with exact answers	6 / 12	12 / 12
Unsafe feeds held	12 / 12	12 / 12
Holds identifying the injected cause	6 / 12	12 / 12
False certifications	0	0

The gain is compatibility with six additional usable exports. Both methods safely held all unsafe feeds. These are synthetic exports from predesigned layout/error families, not an evaluation of arbitrary real merchant schemas.

Four-market training comparison

Every model scored the same 4,865 product contacts, including 679 retained-basket positives. Financial arms made their own campaign choices within the same frozen markets. Feature selection used whole-market validation and equal search budgets for the statistical and ontology selectors.

Training policy	ROC AUC	Log loss	PayPal net	Purchases
Pooled current model	0.8271	0.3236	$14,968.81	2,837
Product rows, all eligible fields	0.8479	0.3034	$15,001.09	2,845
Product rows, statistical selector	0.8510	0.3019	$15,019.54	2,845
Product rows, ontology selector	0.8524	0.3012	$15,040.62	2,853

Much of the quality gain comes from matching rows to the task. Ontology selection added **$21.08 PayPal net**, **$182.49 merchant contribution** and **eight purchases** over conventional feature selection across these four markets. It lost $11.07 PayPal net versus the pooled model on the reactivation seed. The small aggregate advantage is not a universal improvement.

Evidence: mapping protocol, training methods and per-market results, full training scores.

CHAPTER 18

Fresh planner versus planner with ontology

This is a **separate one-market study** on the same judge seed. Both AI arms made six fresh Gemini decisions, fitted two models, tested campaigns in three development markets and used the same financial verifier. The first five decisions used Flash-Lite; Pro handled the final review.

Policy	PayPal net after AI	Merchant contribution	Purchases
Fixed rules	$3,309.83	$38,928.95	627
Fresh planner	$3,391.08	$39,494.31	642
Fresh planner + ontology	$3,424.78	$39,953.81	643

Inference estimates were **$0.034568** for the planner and **$0.042875** for the ontology-assisted planner. Their sum, **$0.077443**, covers 12 new provider responses. This study created zero native PayPal payments.

What actually changed

The plain planner used a 28-day window, trained a shopper-intent forest and a response model, and proposed intent assistance. It increased contact intensity after a positive trial. The verifier replaced its preferred forest with the response model.

The ontology planner also chose a 28-day window. It requested eight pre-contact fields: interest age, category affinity, recency, prior purchases, price, stock, earlier response rate and mature retention rate. That narrow model underperformed the broader model and was not deployed.

Its first ranked-discovery proposal changed supply and used a 10-per-100 contact ceiling. The stress test lost money. The LLM restored the incumbent supply settings and reduced the ceiling to six per 100. The revision passed and used the broader 20-field model.

What the comparison supports

The ontology-assisted workflow added **$33.70 modeled PayPal net**, **$459.50 merchant contribution** and **one purchase** over the fresh planner. The difference combines data, model and campaign choices; it does not isolate ontology as the sole cause. The successful revision and rejection of the weak feature experiment are the concrete decision evidence.

The $3,424.78 total differs slightly from the featured campaign's $3,424.79 because the fresh ontology review had a different inference cost. Do not merge the two executions or their costs.

Evidence: registered fresh comparison, complete decisions and results.

CHAPTER 19

Does the improvement repeat?

The reactivation replication registered six fresh seeds before paid decisions. After 30 days of organic shopping, one planner and a three-role team saw the same snapshot. Each had six decisions, two model experiments, identical campaign choices and the same financial verifier. All campaigns were locked before later purchases were evaluated.

Aggregate difference across six markets	PayPal net after AI	Merchant contribution	Purchases
Planner + verifier minus fixed	+$809.18	+$8,341.39	+135
Team + verifier minus fixed	+$635.39	+$6,514.36	+101
Team + verifier minus planner	-$173.79	-$1,827.03	-34

The planner improved all three primary outcomes over fixed on **six of six seeds**. The team did so on **five of six**. All 12 reviews completed, with 72 distinct Gemini responses, 24 local fits and zero unpaid liabilities. Estimated study inference was **$0.456471**; no native payments were created.

How much did the verifier add?

Scoring the same planner decisions with and without the additional model check isolated **+$21.04 PayPal net and two purchases** across the six planner markets. The corresponding team check added **+$35.57 and three purchases**. The median incremental PayPal gain from the verifier was zero in each arm.

The verifier changed one single-planner ranking. For the team it allowed one otherwise ineligible campaign and changed another ranking. It often retained an already eligible choice. This is a bounded contribution, not a claim that every review receives a large uplift.

Why this matters for architecture

The experiment supports a campaign planner with specialist tools and a verifier as the default. Role separation remains useful in the Studio investigation, where cohort selection and insight editing have distinct artifacts. It does not establish a financial advantage for adding more campaign agents.

Six seeds within one modeled reactivation regime are a repeatability check, not a broad production generalization study. The seed-by-seed table and negative comparisons remain available in the evidence.

Evidence: replication report, locked decisions, paired results.

CHAPTER 20

Scaling to a larger shared market

The separately versioned **commerce-metropolis-1** ladder uses 50 shoppers per merchant. Each tier is one shared market with inventory competition, request allocation and supplier clearing. Its money parameters remain documented, while population and stock/cash normalization change with size.

Merchants / shoppers	Fixed PayPal net	Planner + ontology	PayPal gain
96 / 4,800	$24,395.05	$24,418.68	+$23.63
192 / 9,600	$49,530.34	$51,406.77	+$1,876.43
384 / 19,200	$100,111.95	$103,422.70	+$3,310.75

Merchants	Merchant contribution change	Extra retained purchases	PayPal gain / 100 shoppers
96	-$480.29	+1	$0.49
192	+$18,112.09	+314	$19.55
384	+$36,042.88	+532	$17.24

What the agent did at 384 merchants

The day-31 ontology experiment trained an eight-feature product-contact model. Its recommendation AUC was **0.7620**, versus **0.7503** for the broader candidate. A revised intent-assistance campaign passed the proxy verifier and deployed that model. At day 61 the planner proposed alternatives, but the verifier retained the incumbent.

The run used 12 fresh model decisions, four local fits and **$0.059695** estimated inference. It settled through day 126 with zero unpaid liabilities and zero native payments. The renderer samples shopper sprites and pages merchants in groups of 32; all shoppers remain in the economic engine.

Limits of the scale claim

There is one seed per size, with fresh planning at each size. The verifier uses three 12-merchant / 600-shopper proxy markets; the final fixed comparison executes the full market. Larger absolute gains combine size and policy effects. The 96-merchant arm harmed merchant contribution, and per-shopper gain did not rise monotonically.

The hosted free-run interface is capped at 12 merchants / 600 shoppers. Large results are available as archived evidence; fresh large runs require configured local execution.

Evidence: scale assumptions and attempts, run records and costs.

CHAPTER 21

Local setup and repository map

The saved judge walkthrough runs without provider credentials or Python. Install **Node.js 22.16 or newer**, then run:

```
git clone https://github.com/VinceVence/market-metro.git
cd market-metro
npm ci --ignore-scripts
npm start
```

Open **http://127.0.0.1:4173/judge.html** for the walkthrough or **/landing.html** for the product entry. The private-package flag in **package.json** prevents npm publication; it does not make the GitHub repository private.

Optional local numerical execution

Use the pinned Python dependencies for new numerical simulations, training or saved-decision scoring:

```
uv venv .local/marketing-venv --python 3.12 \
  --cache-dir .local/uv-cache
uv pip install --python .local/marketing-venv/bin/python \
  --cache-dir .local/uv-cache \
  -r scripts/marketing/requirements.txt
npm run test:judge-simulation
```

The numerical dependencies are **NumPy 2.3.4** and **scikit-learn 1.7.2**. Fresh Gemini and native PayPal modes need private operator configuration. Copy the blank **.env.example** for Sandbox credentials only; never put secrets in browser code, exports or source control. The optional configured provider paths can incur costs or mutate Sandbox state.

Path	Responsibility
demo/	Browser UI, market, judge flow and portable evidence
scripts/serve.mjs	Local / hosted HTTP entry
scripts/commerce-expanded/ harness-v2/	Campaign contracts, tools, controller and trials
scripts/ontology/	Feed checks, training contracts and study runners
scripts/studio/	Read-only projections and investigation services
tests/, **docs/**	Verification and experiment evidence
scripts/deployment/, **Dockerfile**	Tracked-file packaging and Cloud Run deployment

Evidence: developer guide, package scripts, Python dependencies.

CHAPTER 22

Cloud Run, security and cost controls

The public service runs in the isolated **supply-yard-pilot-261021** project, **us-central1**, as service **market-metro**. This is a shared Sandbox judging deployment, not a production payment service.

Runtime and data boundaries

- The container runs as an unprivileged user, handles SIGTERM and supplies readiness endpoints.
- Public route allowlists deny fresh campaign execution and payment mutations. Same-origin checks and request/body limits constrain permitted mutations.
- The runtime uses managed identity. No PayPal credentials, saved buyer tokens, ADC, private database or downloaded service-account key enters the deployment archive.
- Agent Studio has a separate model route and a durable **$0.25 shared allowance**. Its investigation can publish views, not payment or campaign records.
- New public markets reuse the saved policy and are capped at 12 merchants / 600 shoppers. One active simulation and four successful runs per instance bound shared use.
- Temporary instance storage may disappear after a restart. Saved bundled evidence is durable in the image; newly generated sessions should be exported.

Deployment procedure

The deployment helper packages tracked runtime paths only, records a source manifest, builds an image and deploys by immutable digest. The source bucket stays private. Runtime identity and build identity have separate privileges.

```
python3 scripts/deployment/package.py
python3 scripts/deployment/cloud-run.py build
python3 scripts/deployment/cloud-run.py status
# After a successful build:
python3 scripts/deployment/cloud-run.py deploy
# After the revision is ready:
python3 scripts/deployment/cloud-run.py publish
```

First-time provisioning and operator authentication are documented separately. These are operator commands, not required judge actions.

Spending and production limits

The service uses zero minimum instances, one maximum instance and request-based CPU billing. Model ledgers and reservations bound provider usage; cloud budget alerts are not hard spending caps. Token cost estimates exclude delayed hosting/build bills. Commercial deployment would additionally require authenticated tenant isolation, durable storage, monitoring, licensing and explicit payment authorization.

Evidence: deployment guide, public endpoint policy, container, publication audit.

CHAPTER 23

Verification and reproducibility

The submission uses checks at distinct levels. Passing unit tests, replaying recorded evidence and observing a native provider receipt are complementary; none substitutes for all the others.

Evidence level	What it checks	Cost / mutation boundary
Submission tests	Navigation, contracts, ontology, accounting and public-service boundaries	No paid model or native payment calls
New-seed saved-policy test	Numerical execution, mapping hold/resume and settlement	Local CPU; saved model decisions
Recorded benchmark scoring	Locked campaigns and model artifacts on paired seeds	Local fitting/scoring; no fresh inference
Native provider proof	Captured Sandbox IDs, states, fees and linked order	Archived prior execution; replay is read-only
Fresh provider run	New model choices or payment lifecycle	Separate configured mode with spend / consent controls

Safe checks for a reviewer

```
npm test
npm run test:judge-simulation
node scripts/ontology/planner-demo.mjs
node scripts/commerce-expanded/harness-v2/score-reactivation.mjs
```

The last command uses the locked six-seed decisions, verifies model artifacts, and writes an independent report under **.local/**. Numerical commands require the Python environment described earlier and may take longer than UI playback.

Accounting and data checks

The test suite covers unique checkout counts, repeated-item-row immunity, exact money reconciliation, missing or conflicting mappings, future-feature rejection, model parity, publication guards, receipt binding and public-mode denial of provider actions. GitHub CI runs submission tests and the saved-policy simulation on Node 22 and Python 3.12.

The November 11 publication audit examined reachable history with Gitleaks and scanned configured credential values without exposing them. The scan recorded zero unresolved findings after reviewing three exact non-secret hash matches. This is a bounded scan, not a security certification. Credentials and runtime state remain outside source control.

Reproduction caution: fresh-model and native Sandbox commands are excluded from this safe list. Historical studies can target different economic versions; use the matching protocol and fingerprints.

Evidence: CI workflow, publication audit, score reproduction.

CHAPTER 24

Limitations and the next validation steps

What the submission establishes

The application demonstrates a functioning path from market events to certified data, local model training, bounded LLM campaign decisions, deterministic publication, shopper outcomes and linked PayPal Sandbox evidence. It also demonstrates native AG Studio agent handoff and verified dashboard publication.

Recorded controlled experiments show gains in the modeled economy, with negative results and denominators preserved. Ontology binding improves compatibility on the defined held-out feed families. Task-specific training improves prediction quality on the documented product-contact evaluation.

What remains unproven

- Production PayPal profit, real merchant ROI and causal customer lift outside the simulator.
- An advantage from more campaign agents: the measured single planner outperformed the tested team in aggregate.
- Universal ontology gains: familiar mappings already work, selected features can be worse, and the fresh ontology comparison changes several decisions together.
- General scale superiority: one seed per large tier and small proxy trials do not establish gains for every merchant, size or market regime.
- Autonomous discovery of arbitrary merchant schemas: the verifier relies on reviewed definitions, declared units and complete-feed assumptions.
- Payment completion for invoices or subscription offers that remain unpaid or unapproved.

Practical next steps

1. Validate feed definitions and completeness against consented merchant exports, with independent control totals.
2. Calibrate behavior and costs from data available to the intended operator; document the resulting economic version.
3. Pre-register additional scenarios and multiple seeds at each scale before comparing policies.
4. Separate feature, campaign and verifier changes with controlled ablations; report uncertainty and merchant-level downside.
5. Validate real incremental impact with consented experiments before making commercial ROI claims.

Submission and licensing scope

Original code is MIT licensed and the source repository is public. Third-party libraries and assets retain their own terms. AG Studio evaluation notices remain visible; no configured trial key is claimed. The PDF complements the runnable demo, source and video; it does not substitute for competition submission fields or establish sponsor licensing clearance.

Evidence: third-party notices, MIT license, documented study limits, deployment limitations.

CHAPTER 25

Evidence index

This guide describes the public application and evidence snapshot at commit **eb93bac**, dated **11 November 2026**. Repository links in the PDF point to that snapshot so the described results remain auditable as the project develops.

Primary evidence bundles

Question	Recorded evidence
What happened in the featured campaign?	Judge campaign, frames, policy deltas and receipt
What does the complete warehouse show?	Portable judge intelligence
Does the mapping hold unseen formats?	24-feed ontology evaluation
Does training-data selection matter?	Four-market feature study
Did fresh ontology planning improve this run?	Three-way fresh campaign comparison
Do reactivation gains repeat?	Six-seed paired financial results
What happens at larger scale?	96 / 192 / 384 merchant run records
How does the Studio agent handoff work?	Native-tool investigation trace
Which PayPal lifecycle actions completed?	Service states and verification

Reading the evidence correctly

Preserve the run ID, seed, accounting period, action cohort, economic version and inference cost when quoting a result. The guided campaign, fresh ontology campaign, feature ablation, large-market run and Studio investigation are separate executions. Their counts and gains must not be summed into one claimed customer outcome.

CHAPTER 26

Useful links and credits

Open the submission

Resource	Link
Product landing page	Open Market Metro
Guided judge walkthrough	Market, agent decisions and impact
Agent Studio	Query, verify and publish
Intelligence	Models, decisions, financials and ontology
Public source	VinceVence / market-metro
Project media	Gallery and diagrams
Local and hosted testing	Judge instructions

Technology credits

Layer	Tools
AI planning	Gemini 3.1 Flash-Lite and Gemini 3.1 Pro Preview on Vertex AI
Local machine learning	Python, NumPy and scikit-learn
Commerce integration	PayPal Sandbox REST APIs
Intelligence workspace	AG Studio 3.0.0, AG Grid and AG Charts
Environment and storage	Three.js, Node.js and SQLite
Deployment	Google Cloud Run
Project-owned controls	Ontology, typed execution contracts, financial verifier and spend ledger

NVIDIA Auto Ontology informed the ontology design. It is not an installed dependency. The active candidate uses authored products and customer behavior; Channel3 is not part of the current runtime.

Author: Lanz Vincent Vencer. **Project:** Market Metro - Agentic Commerce That Learns. **Edition:** Technical submission guide, November 2026. Figures use the existing project gallery and actual application captures; typography uses DM Sans. Original code is MIT licensed; dependencies and assets retain their own terms.

Evidence: package versions, third-party notices, source license.